COMP3070 — Symbolic Artificial Intelligence Coursework 1: Exam Timetabling

Full reference report draft | Integer and Boolean Z3 formulations

Group: [INSERT GROUP LETTER] Members: [INSERT NAMES / STUDENT IDs] Submission: 13 November 2026, 6:00 p.m. (Malaysia time)

REFERENCE DRAFT — review, substantiate and rewrite in the group's own words before submission. Measured results are drawn from the uploaded November 5 full benchmark run; hardware details, group particulars and final source-code commit must be completed by the team.

Sources used: COMP3070 Coursework 1 revised brief; the supplied Python implementations, shared utilities, tests, final results_main.csv and results_scalability.csv; the team's metadata specification; and the Z3 paper and API documentation listed in References.

1. Report contents and introduction

1.1 Problem and motivation

Examination timetabling is a constraint-satisfaction problem in which a finite set of examinations must be allocated to available timeslots and rooms. Every scheduled examination must satisfy assignment, room, capacity and student-clash requirements. The purpose of this coursework is not to optimise a numerical objective such as total timetable length, but to determine whether a feasible assignment exists and to produce one when possible.

Let E, S, T and R denote the finite sets of examinations, students, ordered timeslots and rooms. Each input instance specifies their cardinalities, capacities c(r), and examination-registration pairs (e,s). Our solver returns sat and a list of triples (exam,slot,room) if the constraints can all be satisfied; it returns unsat if no such assignment exists. Internally, the Z3 API may also return unknown, which is retained rather than incorrectly treated as unsat.

1.2 Objectives and contributions

The group implemented two independently encoded, comparable Z3 models: (i) an integer formulation with a slot and room integer for each exam; and (ii) an alternative Boolean formulation with a binary decision for each possible exam–slot–room assignment. Both share input parsing and metadata validation, and both implement the eight explicitly named constraints C1–C8 used in the code.

- End-to-end command-line handling of the official .txt instance format, including sat/unsat, a timetable for sat, and solving time in milliseconds.
- Two additional metadata-based restrictions: facility compatibility and duration fit, represented in optional companion .extra.json files while preserving original text inputs.

- An independent Python timetable validator, boundary/error tests, random tiny-instance comparison with exhaustive allocation search, and repeated benchmarks.
- An integer-model blocking-clause method to establish the existence or absence of a second distinct complete timetable.
- Head-to-head correctness, solve-time, model-size and scalability comparisons, with exported CSVs and reproducible chart-generation code.

1.3 Scope and important qualification

For reporting continuity, the implementation names C1–C5 "main" and C6–C8 "additional". The coursework brief, however, lists "exactly one room and exactly one slot" together as a single numbered main requirement. A stricter mapping would combine C1/C2 and use C6 as the fifth main rule, leaving C7/C8 as two explicitly distinct extensions. Therefore, the five-main-plus-three-additional categorisation is a rubric-interpretation risk. It should be clarified with the convenor or mitigated by introducing and validating another genuinely independent extension before final submission. This draft does not claim that ambiguity is resolved.

2. Mathematical formulation I — integer variables

2.1 Sets, input parameters and conventions

Symbol	Meaning
$E=\{0,\dots,nE-1\}$	Exam IDs; every exam requires one complete assignment
$S=\{0,\dots,nS-1\}$	Student IDs
$T=\{0,\dots,nT-1\}$	Ordered timeslot IDs; adjacency means successive indices
$R=\{0,\dots,nR-1\}$	Room IDs
$A \subseteq E\times S$	Distinct exam/student registration relation
$N(e)=\{s\in S:(e,s)\in A\}$	Students enrolled in exam e
$Q(s)=\{e\in E:(e,s)\in A\}$	Exams taken by student s
$c(r)$	Capacity of room r
$F(e)$, $G(r)$	Required facilities for exam e; facilities offered by room r
$d(e)$, $\ell(t)$	Exam and slot durations, in integer minutes

Duplicate registration pairs are deduplicated when computing enrolment counts. The original Moodle files contain no facilities or durations. For those cases, the program labels the data as baseline and inserts synthetic neutral values: empty facility requirements, empty room facility sets, 60-minute exams and 120-minute slots. Consequently C7 and C8 are nonbinding on the original instances; these figures are implementation assumptions, not externally supplied facts.

2.2 Decision variables and domains

For each exam e, create two integer decision variables: τe is the chosen slot and ρe is the chosen room. The vector (τ,ρ) completely describes a timetable; no separate assignment indicator is required.

$$\tau_e \in \{0,\dots,nT-1\}, \quad \rho_e \in \{0,\dots,nR-1\} \quad \text{for each } e\in E$$

If an exam exists while no slot or no room exists, the domain bounds make the model unsatisfiable. With zero exams, the empty timetable is satisfiable.

2.3 Integer constraint set

C1 and C2 — exactly one slot and exactly one room

The single integer value of τe gives one timeslot (C1) and the single integer value of ρe gives one room (C2). Domain restrictions rule out unavailable values. These are logically separate requirements but arise from one combined assignment decision per exam.

$$\forall e \in E: 0 \leq \tau_e < nT \quad \wedge \quad 0 \leq \rho_e < nR$$

C3 — no room double-booking

Any two distinct exams may share a slot or share a room, but they must not share both.

$$\forall e,f \in E,\ e<f: (\rho_e \neq \rho f) \vee (\tau_e \neq \tau f)$$

The implementation adds one z3.Or expression for each unordered exam pair.

C4 — room capacity

Assigning an exam to a room is permitted only when the room capacity is at least the number of distinct students enrolled in that exam.

$$\forall e \in E: |N(e)| \leq c(\rho_e)$$

Since room capacities and enrolments are fixed input data, the code filters inadmissible room IDs: if $c(r) < |N(e)|$ then assert $\rho e \neq r$.

C5 — prohibit consecutive student examinations

For any two different exams attended by the same student, the timeslot indices must not differ by exactly one. This is a separation rule, not a simultaneous-clash rule.

$$\forall s \in S,\ \forall e<f \in Q(s): |\tau_e - \tau f| \neq 1$$

In Z3, the integer formulation uses two inequalities $\tau e - \tau f \neq 1$ and $\tau f - \tau e \neq 1$ for each unique shared-student exam pair.

C6 — prohibit simultaneous student examinations

A student cannot sit two exams in the same slot, even if two different rooms would otherwise be available.

$$\forall s \in S,\ \forall e<f \in Q(s): \tau_e \neq \tau f$$

C5 alone does not reject same-slot exams (difference zero); C6 alone does not reject adjacent slots (difference one). Together they imply $|\tau e - \tau f| \geq 2$ for exams sharing a student. Neither rule renders the other redundant.

C7 — required facilities

A room is eligible only if it has every facility required by the assigned exam. Facility matching is set inclusion and case-sensitive.

$$\forall e \in E: F(e) \subseteq G(\rho_e)$$

For each incompatible room r with $F(e) \not\subseteq G(r)$, the integer model asserts $\rho e \neq r$. More facilities than requested are acceptable.

C8 — duration fit

Each exam must fit wholly within one chosen timeslot; equality of exam duration and timeslot duration is allowed. Exams cannot be split across slots.

$$\forall e \in E: d(e) \leq \ell(\tau_e)$$

If $d(e) > \ell(t)$, the integer formulation asserts $\tau e \neq t$. Timeslots are ordered by index for C5, regardless of their duration values.

2.4 Z3 solving and second-solution detection

The solver builds all constraints, then calls solver.check(). For sat, model.eval extracts a concrete τe and ρe for each exam. For unsat, no timetable is returned. The measured solve_time_ms covers the first solver.check() only, while build_time_ms measures preprocessing and assertion construction. A second satisfiability check is optional and excluded from the primary timing.

To determine whether another complete timetable exists, the integer solver first saves one satisfying model (τ^*, ρ^*). It then temporarily asserts a disjunction requiring at least one decision variable to differ from that model:

$$\vee_{e \in E} [(\tau_e \neq \tau^*_e) \vee (\rho_e \neq \rho^*_e)]$$

If the second check returns sat, a distinct feasible allocation exists; if it returns unsat, the original allocation is unique with respect to all exam-slot-room decisions. If the second check returns unknown, uniqueness is not established. The implementation uses solver.push()/solver.pop() so the extra clause is temporary. With no exams, only the empty timetable exists.

3. Alternative formulation II — Boolean assignment variables

3.1 Variables and relationship to the integer model

The alternative model represents each possible assignment explicitly. For every exam e, slot t and room r, create $X[e,t,r] \in \{False, True\}$. True means that exam e is scheduled in slot t and room r. Its declared variable count is $nE \times nT \times nR$, rather than the integer model's $2 \times nE$. Both encodings describe the same intended set of feasible timetables under the same inputs.

$$X_{e,t,r} = 1 \Leftrightarrow \text{exam e occupies timeslot t and room r}$$

For convenience, define Y[e,t] as a derived Boolean expression indicating that exam e occupies timeslot t (logical OR over rooms), and B[s,t] as the logical OR of Y[e,t] for all exams registered by student s. These are composite Z3 formulas rather than additional declared decision variables.

$$Y_{e,t} = \vee_{r \in R} X_{e,t,r}; \qquad B_{s,t} = \vee_{e \in Q(s)} Y_{e,t}$$

3.2 Boolean constraint set

C1/C2 — exactly one assignment pair per exam

Exactly one Boolean variable for each exam is true over all available slot–room pairs. This single cardinality equality simultaneously ensures one timeslot and one room.

$$\forall e \in E: \Sigma_{t \in T} \Sigma_{r \in R} X_{e,t,r} = 1$$

The implementation uses z3.PbEq with unit weights. If there are no possible pairs for an existing exam, it adds False, correctly making the instance unsatisfiable.

C3 — room cannot contain overlapping exams

$$\forall t \in T, \forall r \in R: \Sigma_{e \in E} X_{e,t,r} \leq 1$$

The implementation uses z3.AtMost for each room–slot pair when there are at least two exams.

C4 — room capacity

$$\forall e,t,r \text{ with } |N(e)| > c(r): \neg X_{e,t,r}$$

Invalid room assignments are explicitly forbidden instead of invoking numerical capacity expressions over chosen integers.

C5 — no consecutive exams for one student

$$\forall s \in S,\ \forall t \in \{0,\ldots,nT-2\}: \neg(B_{s,t} \wedge B_{s,t+1})$$

The implementation builds busy[t] for each student and adds z3.Not(z3.And(busy[t],busy[t+1])) for adjacent slots. This forbids back-to-back occupied slots, even when the exams are different.

C6 — no simultaneous exams for one student

$$\forall s \in S, \forall t \in T: \Sigma_{e \in Q(s)} Y_{e,t} \leq 1$$

The model uses z3.AtMost over all of a student's registered exams for each slot. It thereby rejects simultaneous exams irrespective of the assigned rooms.

C7 — facility compatibility

$$\forall e,t,r \text{ with } F(e) \not\subseteq G(r): \neg X_{e,t,r}$$

A Boolean assignment to a facility-incompatible room is set to false. The eligibility calculation uses ordinary Python sets of static metadata.

C8 — duration compatibility

$$\forall e,t,r \text{ with } d(e) > \ell(t): \neg X_{e,t,r}$$

Assignments to slots that are too short are explicitly ruled out; exact fits remain permitted.

3.3 Correctness intuition and encoding trade-offs

For any valid integer assignment ($\tau e, \rho e$), set exactly X[e,τe,ρe] to true for each exam and every other X to false. Every Boolean constraint then holds because the timetable satisfies the corresponding domain, clash, capacity, student and compatibility requirements. Conversely, exact-one in the Boolean model selects precisely one (t,r) pair per exam, which yields a corresponding integer assignment. Under identical input metadata and constraints, the feasible timetables therefore correspond one-to-one between the models.

The integer encoding is compact in declared variables and expresses slot differences directly. The Boolean encoding makes eligibility and occupancy explicit, with convenient pseudo-Boolean equalities and at-most-one rules. Its larger assignment space may create additional solver work as the number of timeslots and rooms increases, but declared variable count alone does not determine total solver complexity or runtime.

4. Implementation

4.1 Program structure and execution

Component	Responsibility
Formulation1.py	Integer decision variables, C1–C8 Z3 constraints, first-model extraction, optional alternative-solution check and CLI.
Formulation2.py	Boolean X[e,t,r] assignments, C1–C8 Z3 constraints, model extraction and CLI.
shared_input.py	Instance representation, text parser, companion JSON loading, consistent metadata normalization, input validation and enrolment indexes.
timetable_check.py	Independent ordinary-Python verification of returned SAT timetables.
tests/test_*.py	Constraint, input, edge-case, extended-metadata, randomized exhaustive and regression tests.
tests/compare_formulations.py	Repeat benchmarks, cross-formulation checks, scalability generation, CSV/Markdown/chart outputs.
tests/alt_solutions_check.py	Calls Formulation 1 with blocking-clause checking for each official SAT instance.

From the repository root, after installing requirements.txt, a user can run:

```
python Formulation1.py instances/sat1.txt
python Formulation2.py instances/sat1.txt
python -m unittest discover -s tests -t . -p "test_*.py" -v
python tests/compare_formulations.py --f1 . --f2 . --out results
python tests/alt_solutions_check.py --f1 .
```

Both CLIs print the status first and, for sat, the timetable triples. They also print solver.check() time in milliseconds, model-construction time, declared variable counts and top-level assertion counts. The CLI and solver functions are separate: solve_integer and solve_boolean return dictionaries suitable for automated testing and benchmarking.

4.2 Data processing and metadata assumptions

The shared parser reads the standard .txt header fields and capacity lines followed by zero-based (exam_id,student_id) registrations, preserving the supplied template convention. If a same-stem .extra.json companion file exists, it loads all four metadata arrays: exam_requirements, room_facilities, exam_durations and slot_durations. Their lengths are checked against exam, room and slot counts. Malformed present metadata raises a value or file error; it is not misreported as unsat.

On baseline files, synthetic neutral metadata is inserted so C7/C8 do not alter the original C1–C6 feasibility. On extended files, the metadata can make an otherwise feasible baseline case

unsatisfiable. The --baseline CLI flag deliberately ignores companion metadata and permits direct comparison. Facility matching is case-sensitive, and all durations must be positive integer minutes.

4.3 Independent validation and test design

After each sat result, timetable_check.validate_timetable checks for exactly one triple per exam, valid IDs, absence of room double-booking, sufficient room capacity, no simultaneous or consecutive examinations for any student, and appropriate facilities and slot duration. The validator uses ordinary Python rather than the solver's Z3 expressions, reducing the risk of verifying a model with an identical logical mistake.

The test suites cover the 20 official instances, eight synthetic extended examples, individual constraint boundaries, malformed registration IDs and metadata, zero-exam and zero-room cases, duplicate enrolments, and two families of 48 small generated examples checked against independent exhaustive enumeration. The latest integrated suite previously contained 73 test methods; the exact count and green run should be reconfirmed and recorded from the final commit.

4.4 Performance measurement and reproducibility

The final comparison run measured each supplied instance ten times per formulation and reported mean, median and population standard deviation of the first solver.check() call, separately from model construction. Generated scalability instances used three fixed random seeds and three solve repetitions per seed at each size, with a 20,000 ms timeout. The script performs a warm-up solve and exports raw results, a Markdown summary and three figures. For the original 20 cases, the benchmark explicitly requests baseline mode; for the eight examples, it loads extended metadata.

The supplied individual formulation CSVs record Python 3.14.8 and Z3 5.3.3; the full comparison summary records Z3 5.3.3. Fill in the actual CPU, RAM, OS, date and source-code commit before submission: [HARDWARE DETAILS / OS / COMMIT]. No hardware specifications have been inferred from the uploaded files.

5. Evaluation

5.1 Correctness on supplied and extended cases

Table 1 reports the complete final full-run dataset. "F1/F2 status" is the observed satisfiability result; "time" is the mean of ten solver.check() measurements in milliseconds. The expected labels come from the supplied filenames. Every satisfiable timetable was also checked independently; unsat results have no timetable to validate.

Instance	Mode	Expected	F1	F1 ms	F2	F2 ms
sat1.txt	baseline	sat	sat	0.76	sat	0.20
sat2.txt	baseline	sat	sat	1.55	sat	0.61
sat3.txt	baseline	sat	sat	2.83	sat	0.52
sat4.txt	baseline	sat	sat	0.88	sat	0.13

sat5.txt	baseline	sat	sat	2.73	sat	0.52
sat6.txt	baseline	sat	sat	1.67	sat	2.04
sat7.txt	baseline	sat	sat	2.09	sat	2.62
sat8.txt	baseline	sat	sat	1.60	sat	1.76
sat9.txt	baseline	sat	sat	2.14	sat	4.40
sat10.txt	baseline	sat	sat	3.08	sat	7.76
unsat1.txt	baseline	unsat	unsat	0.69	unsat	0.10
unsat2.txt	baseline	unsat	unsat	0.72	unsat	0.11
unsat3.txt	baseline	unsat	unsat	1.94	unsat	0.78
unsat4.txt	baseline	unsat	unsat	2.37	unsat	0.93
unsat5.txt	baseline	unsat	unsat	2.39	unsat	3.67
unsat6.txt	baseline	unsat	unsat	2.41	unsat	1.82
unsat7.txt	baseline	unsat	unsat	2.11	unsat	0.69
unsat8.txt	baseline	unsat	unsat	2.70	unsat	3.31
unsat9.txt	baseline	unsat	unsat	9.35	unsat	50.00
unsat10.txt	baseline	unsat	unsat	16.47	unsat	517.23
sat_combined_gap.txt	extended	sat	sat	1.27	sat	0.56
sat_duration_exact.txt	extended	sat	sat	0.62	sat	0.18
sat_duration_forced.txt	extended	sat	sat	0.71	sat	0.11
sat_facilities_forced.txt	extended	sat	sat	0.79	sat	0.12
unsat_combined_contention.txt	extended	unsat	unsat	0.79	unsat	0.11
unsat_duration_long.txt	extended	unsat	unsat	0.62	unsat	0.11
unsat_facilities_missing.txt	extended	unsat	unsat	0.62	unsat	0.03
unsat_facilities_partial.txt	extended	unsat	unsat	0.63	unsat	0.09

Across all 28 cases, Formulation 1 and Formulation 2 each matched every filename expectation (28/28) and agreed on status (28/28). Their satisfiable schedules passed independent validation. This establishes correctness on the evaluated cases, not universal correctness against all unseen instances.

5.2 Timing on official instances

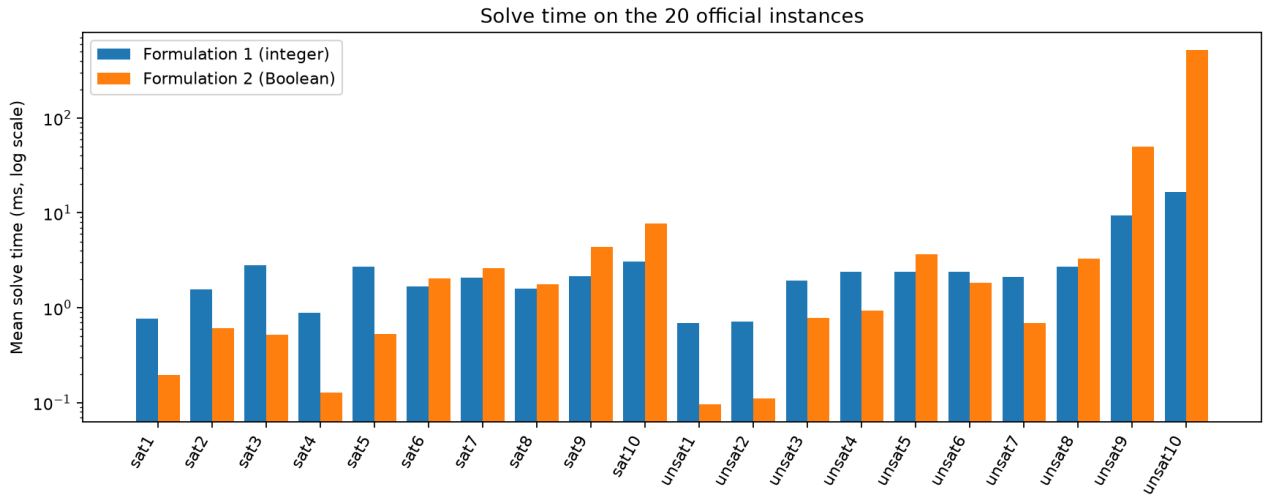


Figure 1. Mean solver.check() time on the 20 official baseline instances; logarithmic vertical axis. Source: uploaded November 5 full-run chart.

On the 20 official cases, Formulation 1 was quicker on nine and Formulation 2 on eleven, using mean solve times. The small cases contain several examples where Formulation 2 finishes in well under 1 ms. On the difficult unsat10 case, the final full run reports 16.47 ms for the integer formulation and 517.23 ms for the Boolean formulation, approximately a 31.4× difference. Unsat9 similarly takes 9.35 ms and 50.00 ms, respectively. These comparisons refer to solve time only; they exclude model construction.

5.3 Declared decision variables and assertion counts

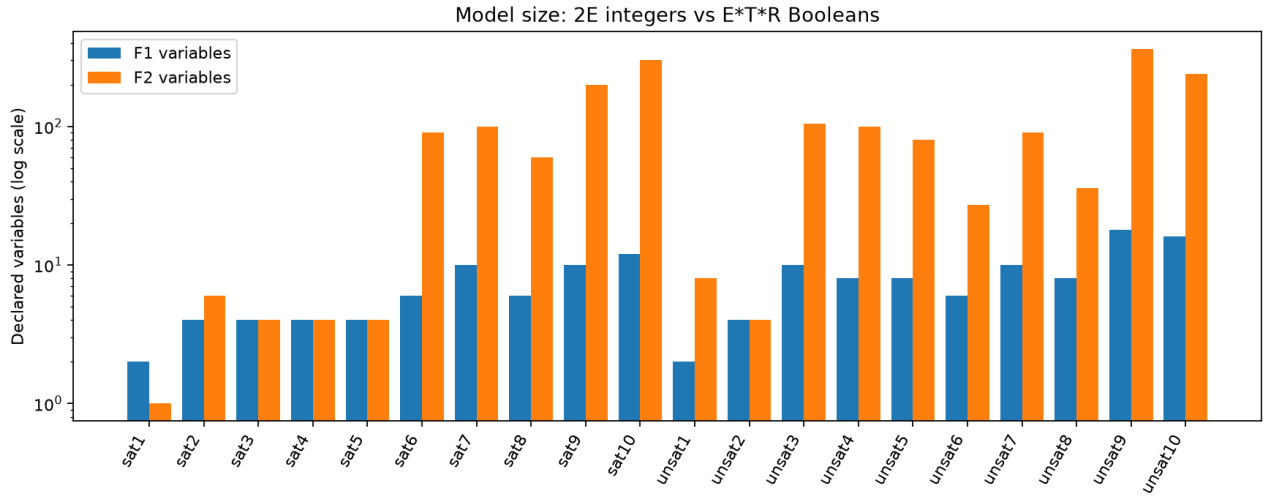


Figure 2. Declared decision variables for both encodings on the 20 official instances; logarithmic vertical axis.

The formal variable-count relationship is 2nE integer variables versus nE×nT×nR Boolean variables. For example, unsat10 has 16 integer assignment variables versus 240 Booleans, while official sat10 has 12 versus 300. There are cases where the Boolean model has equal or fewer declared variables (sat1), so the integer formulation is not uniformly smaller. Top-level Z3

assertion counts are implementation-dependent and should not be confused with internal SAT clauses.

5.4 Scalability study

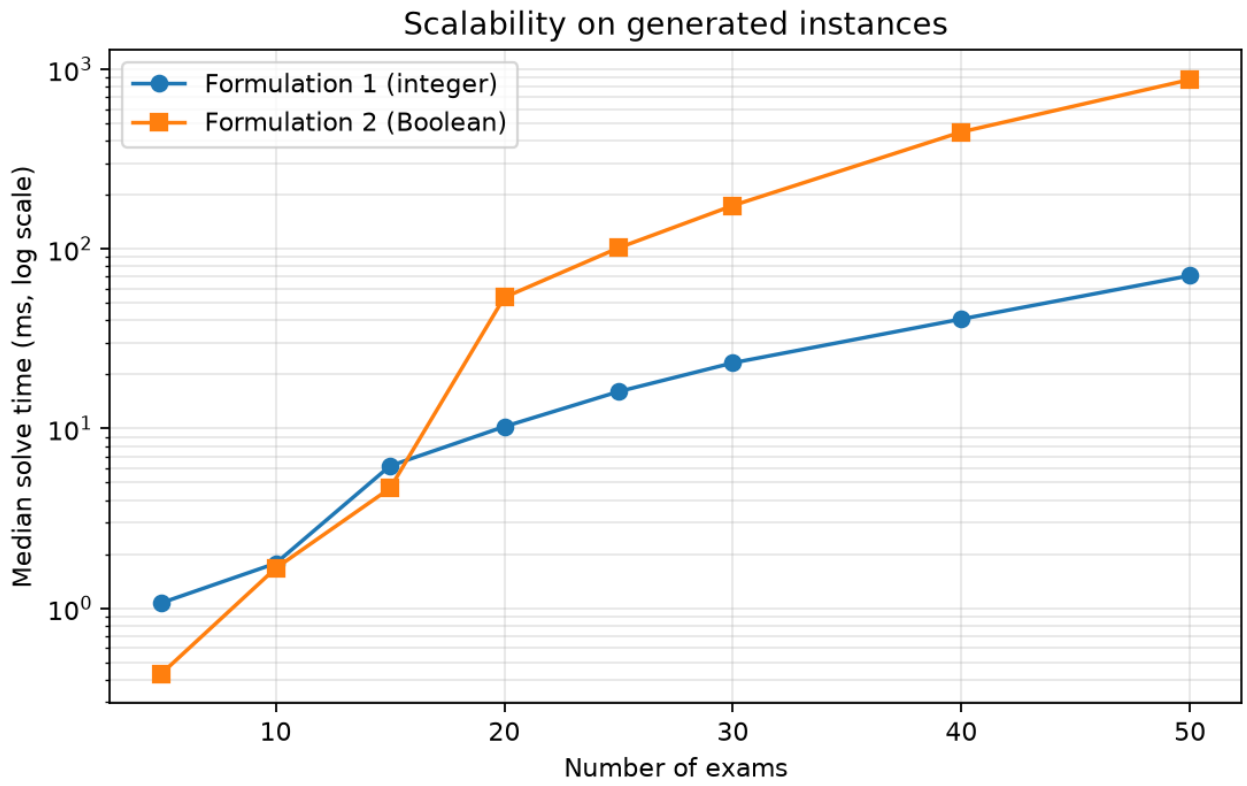


Figure 3. Median solve time versus exam count for generated instances, taking the median over three fixed seeds for each size. Logarithmic vertical axis.

Exams	F1 median ms	F2 median ms	F1 vars	F2 vars
5	1.07	0.43	10	60
10	1.78	1.67	20	240
15	6.20	4.65	30	810
20	10.25	53.71	40	2400
25	16.06	101.34	50	4500
30	23.21	174.24	60	7560
40	40.69	447.48	80	19200
50	70.98	875.66	100	36000

The generated experiments vary the number of examinations through 5, 10, 15, 20, 25, 30, 40 and 50; each size uses seeds 1, 2 and 3. Across all 24 generated cases the two formulations returned the same satisfiability status; fifteen cases were sat and both schedules validated, while nine were unsat. At 20 exams the across-seed median is 10.25 ms (F1) against 53.71 ms (F2); at 50 exams the corresponding values are 70.98 ms and 875.66 ms. These are not independent replicates of exactly the same instance: medians aggregate the three seeded instances, which may differ in SAT/UNSAT outcome.

The generator scales students as twice the exam count, timeslots approximately as 1.2 times the exam count, and rooms approximately as one-quarter of the exam count, with random student registrations and room capacities. Thus the experiment changes multiple dimensions at once. It

uses neutral metadata and therefore evaluates C1–C6 scaling, not restrictive facilities and durations. The sample size is small, and the results should be interpreted as evidence on this generator, not an asymptotic complexity proof.

5.5 Extended C7/C8 examples

The eight examples exercise forced rooms with facilities, missing or partially matched requirements, exact-duration fits, slots too short to hold an exam, and combined restrictions that make instances infeasible. All eight observed statuses match their expected labels for both formulations. In this final run the Boolean formulation was faster on all eight extended examples, but these examples are very small and their measured differences are mostly fractions of a millisecond; they do not generalize to large extended timetables.

5.6 Alternative-solution results

Official satisfiable cases	Result
sat1, sat4	Unique complete timetable under C1–C8 with baseline metadata
sat2, sat3, sat5–sat10	At least two distinct complete timetables

This conclusion is produced by the integer solver's blocking-clause check rather than by counting every timetable. Formulation 2 does not currently perform the equivalent second-model check; this feature difference must be made explicit in any comparison.

6. Discussion

6.1 Interpretation of the empirical difference

The two encodings are logically aligned, as supported by 28/28 matching statuses and the argument in Section 3.3, but they expose different constraints to Z3. Boolean assignments make each potential exam–slot–room combination explicit and allow direct at-most-one and compatibility exclusion. This can be efficient for a very small search space. The integer formulation stores only the chosen room and slot per exam, then relies on comparisons and integer-domain arithmetic; the declaration count grows with exams rather than every combination. This is a plausible explanation for the larger observed runtime gap at 20–50 generated exams, but it is not a controlled causal experiment and does not establish universal superiority.

6.2 Strengths and weaknesses

Aspect	Integer formulation (F1)	Boolean formulation (F2)
Representation	Two bounded integer assignments per exam; compact declared variable count.	Explicit assignment literals; useful for direct eligibility and occupancy rules.
Constraint expression	Slot differences make C5 and C6 straightforward; pairwise exam constraints can grow.	Pseudo-Boolean and AtMost expressions express exactly-one and resource limits naturally.
Observed performance	Strong advantage on larger generated cases and hard official UNSAT cases.	Often faster on tiny supplied and extended cases.
Alternative timetables	Implements second-model blocking check.	No second-model check in current implementation.
Modifiability	Compatible-room and slot restrictions directly remove integer values.	Eligibility restrictions require forbidding corresponding assignment

literals.

6.3 Validity, measurement and methodological limitations

- All official problem statuses are checked using labels implied by filenames, not by an independent formal proof of every UNSAT case. For small generated problems, exhaustive enumeration provides stronger complementary verification.
- Both formulations share one parser and data-preprocessing layer; this ensures consistent comparison but a shared input bug could affect both. Independent timetable checking reduces, but does not eliminate, this risk.
- The baseline examples use synthetic 60-minute exams and 120-minute slots with empty facility requirements; hence they cannot establish the effectiveness of restrictive C7 and C8. The extended suite is much smaller and synthetic.
- Elapsed times are sensitive to Z3 version, Python implementation, system load, CPU power management and process noise. Many sub-millisecond differences should not be overinterpreted.
- Model-construction and solve times are separate. Total end-to-end latency cannot be assessed from solver.check() timing alone, and large Boolean model construction may matter in practical use.
- A 20-second Z3 timeout was configured for scalability, but timeout handling and solver unknown should be explicitly reported if encountered in future experiments.
- The generator varies several input dimensions together and uses three seeds per size; it is not a comprehensive stress test for different density, enrolment patterns, room scarcity or long-duration restrictions.
- Only the integer solver currently reports whether another complete timetable exists. This difference in ancillary functionality does not undermine the SAT/UNSAT comparison but limits feature parity.
- The exact interpretation of five main plus three additional constraints remains a rubric risk until confirmed by the convenor, or until a clearly third independent extension is added.

6.4 Improvements and future work

A stronger study would vary exam count, slot count, room count, enrolment density and metadata difficulty independently; use more random seeds; record complete hardware/OS and Git commit details; analyse both model-construction and solve time; and report confidence intervals or more robust variability summaries. Further modelling work could implement an independently specified third added constraint—such as a configurable maximum number of exams per slot—and mirror alternative-solution checking in the Boolean solver.

7. Conclusion

The project implements two distinct Z3 encodings of the examination timetabling feasibility problem and a common input and validation workflow. Both formulations match the expected SAT/UNSAT classification on all 20 official and eight extended cases, and generated scalability

experiments provide additional comparative evidence. The Boolean encoding performs well on many tiny cases, while the integer encoding scales more favourably on this group's larger random instances, consistent with its much smaller number of declared assignment variables. The independent checker, randomized exhaustive tests, explicit C7/C8 examples, alternative-solution detection and repeatable benchmark generation strengthen the technical evidence.

The strongest remaining deliverable is a carefully justified report: the mathematical model and equivalent constraint semantics must be clear, the performance claims must be restricted to tested conditions, and the five-main-plus-three-additional rule must be addressed transparently. The code and experiments form a solid basis for the requested evaluation, but they do not by themselves prove completeness on unseen instances or determine a final mark.

8. References

de Moura, L., & Bjørner, N. (2008). Z3: An efficient SMT solver. In C. R. Ramakrishnan & J. Rehof (Eds.), Tools and algorithms for the construction and analysis of systems (Lecture Notes in Computer Science, Vol. 4963, pp. 337–340). Springer. https://doi.org/10.1007/978-3-540-78800-3_24

Z3 Development Team. (n.d.). Z3Py API documentation. https://z3prover.github.io/api/html/namespacez3py.html

University of Nottingham Malaysia. (2026). COMP3070 Symbolic Artificial Intelligence: Coursework 1 (Group), revised 3 October 2026 [Coursework brief supplied via Moodle].

Group-developed Python source files, test harnesses, input datasets and results (2026). Unpublished coursework artifacts; include exact repository commit identifier in the final report.

Appendix A. Reproduction commands

From the root directory, with Python and requirements installed:

```
python -m pip install -r requirements.txt
python Formulation1.py instances/sat1.txt
python Formulation2.py instances/sat1.txt
python -m unittest discover -s tests -t . -p "test_*.py" -v
python -m tests.test_formulation1 --benchmark
python -m tests.test_formulation2 --benchmark
python tests/compare_formulations.py --f1 . --f2 . --out results
python tests/alt_solutions_check.py --f1 .
```

The two individual benchmark scripts save F1 and F2 CSV results to results/. The comparison script saves results_main.csv, results_scalability.csv, summary.md and three charts to the folder named in --out. The alternative-solution checker prints results and can be redirected to results/alt_solutions.txt.

Appendix B. Final submission review

- Insert actual group letter, all members and intended contribution acknowledgments.
- Resolve the constraint-classification issue with the convenor or add a separately specified, implemented and tested third extension.
- Record latest test-suite output, Python/Z3 versions, OS/CPU/RAM, data collection date and Git commit.
- Confirm the figures and numeric table are regenerated from the same final revision and benchmark run.
- Review the equations and explanation against both final solver files and apply the team's academic-integrity rules.
- Check the PDF report, references, and code ZIP against the exact Moodle submission instructions.